

Task 1

Visualization Library Documentation: Matplotlib and Seaborn

Library Overview

1. Matplotlib

Matplotlib: A Core Python Visualization Library

Matplotlib is one of the most powerful and widely used data visualization libraries in Python. Developed by John D. Hunter, it supports the creation of a diverse range of static, animated, and interactive plots. It acts as the foundational plotting engine for many other libraries like Seaborn, pandas plotting, and Plotnine, solidifying its role in the Python data science ecosystem. Highly customizable, Matplotlib gives users full control over elements such as figure size, color schemes, font styles, axes, and legends. It integrates well with NumPy and pandas, and supports multiple backends for exporting visuals in formats like PNG, PDF, SVG, and JPG—useful for academic and professional reporting. Matplotlib can create various plot types, including line plots, bar charts, histograms, scatter plots, pie charts, error bars, stack plots, and even 3D plots through `mpl_toolkits.mplot3d`. It's extensively used for exploratory data analysis (EDA), scientific research, and dashboard development—especially in Jupyter notebooks and GUI frameworks like Tkinter. Despite having a steeper learning curve compared to higher-level libraries such as Seaborn, Matplotlib offers unmatched precision. Features like subplotting (`plt.subplot()`), LaTeX rendering for text, and real-time plotting enhance its versatility. Strong documentation and an active community make it a reliable tool for both academic and industry use.

2. Seaborn

Seaborn is a powerful Python library built on top of Matplotlib, designed to simplify the creation of attractive and informative statistical graphics. Created by Michael Waskom, it offers a clean and modern interface that allows users to generate beautiful plots with minimal code. Seaborn works seamlessly with pandas DataFrames, enabling users to directly pass datasets into its plotting functions. It is particularly useful for exploring relationships between multiple variables, providing built-in features for grouping,

aggregation, and statistical estimation. The library supports a wide range of visualizations such as line plots, bar plots, scatter plots, box plots, violin plots, histograms, KDE plots, heatmaps, and pair plots. Many of these visualizations come with useful statistical enhancements—for example, bar plots can show confidence intervals by default, and scatter plots can include regression lines to help analyze trends using functions like `sns.regplot()` or `sns.lmplot()`. Additionally, Seaborn includes various built-in themes and color palettes that improve the visual appeal and readability of charts, making it a popular choice for dashboards, academic presentations, and exploratory data analysis. What makes Seaborn stand out from Matplotlib is its simplicity and declarative style; for instance, creating a boxplot of total bills by day in a restaurant dataset can be done effortlessly with `sns.boxplot(x='day', y='total_bill', data=tips)`. It also enables easy multi-variable comparisons through functions like `sns.pairplot()` and `sns.heatmap()`, which help uncover patterns and correlations with minimal effort. While Seaborn simplifies many Matplotlib functionalities behind the scenes, it still allows advanced users to customize plots at the Matplotlib level. Overall, Seaborn offers a great balance of functionality, ease of use, and aesthetic quality, making it an essential tool for data analysts and scientists working with Python.

Graph Types with Examples

1. Matplotlib

a. Line Plot

A line plot visualizes data by connecting a series of points with straight line segments, making it an excellent choice for showing trends across intervals or continuous variables. The horizontal (x) and vertical (y) axes represent different variables, and the connected points clearly demonstrate how one variable changes in relation to the other over time or sequence. Line plots can display multiple lines simultaneously, allowing easy comparison between different data sets within the same graph. They also offer a wide range of customization options, including colors, markers, line styles, labels, and gridlines, enhancing both clarity and aesthetic appeal. Because of this flexibility, line plots are especially valuable for illustrating time series data, tracking trends, and identifying patterns across various variables over a period.

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4]
```

```
y = [10, 20, 25, 30]
```

```
plt.plot(x, y)
```

```
plt.title("Line Plot")
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.show()
```

b. Bar Chart

A bar chart is a popular visualization method in Matplotlib used to display categorical data through rectangular bars, where the height or length of each bar reflects the value of the corresponding category. These charts make it easy to compare quantities across different groups, highlighting which categories have higher or lower values at a glance. Bar charts can be drawn vertically using `plt.bar()` or horizontally with `plt.barh()`, offering flexibility in presentation. Matplotlib also provides extensive customization options, allowing users to adjust colors, bar widths, alignment, labels, and even add value annotations directly on the bars. This versatility makes bar charts particularly useful for visualizing survey results, sales data comparisons, or any scenario where differences between distinct categories need to be clearly communicated.

```
categories = ['A', 'B', 'C']
```

```
values = [10, 24, 36]
```

```
plt.bar(categories, values)
```

```
plt.title("Bar Chart")
```

```
plt.show()
```

c. Histogram

A histogram is a type of plot in Matplotlib that visualizes the distribution of numerical data by grouping continuous values into bins, or intervals, and displaying the count of data points within each bin. Unlike bar charts, which represent distinct categories, histograms focus on continuous data, with the x-axis showing the bin ranges and the y-axis indicating the frequency of data points in each interval. This makes histograms particularly valuable for understanding the shape, spread, and central tendencies of a dataset, such as identifying normal distributions, skewness, or outliers. Using the `plt.hist()` function, users can customize features like the number of bins, colors, edge styles, and choose whether to display normalized data. Due to their ability to reveal underlying patterns in data, histograms are a key tool for exploratory data analysis and gaining statistical insights.

```
import numpy as np
data = np.random.randn(1000)

plt.hist(data, bins=30)
plt.title("Histogram")
plt.show()
```

2. Seaborn

a. Line Plot

A line plot is an effective and visually appealing way to illustrate trends in data over a continuous variable, such as time or measurement intervals. Seaborn, built on top of Matplotlib, offers the convenient `sns.lineplot()` function, which automatically manages statistical estimation and displays confidence intervals. Typically, the x-axis represents the independent variable, while the y-axis shows the dependent variable. What makes Seaborn's line plot especially powerful is its ability to incorporate grouping through parameters like `hue`, `style`, and `size`, enabling easy comparison of multiple categories within a single plot. Additionally, it smooths noisy data by applying built-in estimators such as the mean or median, providing a clearer view of underlying trends. This

combination of features makes Seaborn line plots ideal for analyzing trends, comparing groups, and visualizing time series data with clarity and elegance.

```
import seaborn as sns
import pandas as pd

df = pd.DataFrame({
    'x': [1, 2, 3, 4],
    'y': [10, 20, 25, 30]
})

sns.lineplot(data=df, x='x', y='y')
```

b. Bar Plot

A Seaborn barplot is a versatile and insightful visualization used to display the average—or another estimator—of a numerical variable across different categories. Unlike basic bar charts that show raw values, `sns.barplot()` aggregates the data, typically plotting the mean for each category along with confidence intervals by default. In this plot, the x-axis represents categorical variables, while the y-axis shows the aggregated numerical values. Barplots are especially useful for comparing group-wise statistics such as average scores, sales figures, or survey responses. Seaborn enhances flexibility by allowing customization through parameters like `hue` to compare multiple groups, `ci` to control confidence interval display, and `estimator` to specify different aggregation functions, such as the median instead of the mean. This combination makes Seaborn barplots a powerful tool for highlighting comparisons and extracting statistical insights from grouped data.

```
df = pd.DataFrame({
    'category': ['A', 'B', 'C'],
    'value': [10, 20, 15]
})

sns.barplot(x='category', y='value', data=df)
```

c. Histogram / Distplot

The Seaborn distplot function was once a widely used tool for visualizing the distribution of continuous data by combining a histogram with a kernel density estimate (KDE) curve. This dual representation allowed users to observe both the frequency of data within intervals and the smoothed probability density function, offering valuable insights into the shape, spread, and modality of a dataset. However, as of Seaborn version 0.11.0, the `sns.distplot()` function has been deprecated. In its place, Seaborn now encourages the use of two separate, more specialized functions: `sns.histplot()` for creating histograms (with optional KDE overlays) and `sns.kdeplot()` for standalone KDE visualizations. This change provides greater flexibility and clearer function responsibilities, aligning with Seaborn's aim for a more intuitive and maintainable API.

```
data = np.random.randn(1000)

sns.histplot(data, kde=True)
```

c. Boxplot

A Seaborn boxplot is a powerful statistical visualization used to display the distribution, variability, and outliers of numerical data across different categories. It summarizes key statistical measures—including the minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum—using a compact box-and-whisker format. The box represents the interquartile range (IQR), with a line marking the median, while the “whiskers” extend to the data range excluding outliers, which are plotted as individual points. The `sns.boxplot()` function in Seaborn makes creating boxplots straightforward and allows for grouping by categorical variables using the `x`, `y`, and `hue` parameters, making it easy to compare multiple groups or subgroups. Boxplots are especially useful for identifying skewness, variability, and potential outliers in the data. Common use cases include comparing student performance across different classes, analyzing income levels across regions, or summarizing any grouped numerical data in a clear and statistically informative manner.

```
df = sns.load_dataset("tips")

sns.boxplot(x="day", y="total_bill", data=df)
```

